# V-OS

Ein grafisches Betriebssystem für den Amstrad CPC 6128

Handbuch

Erstellt aus docs/manual/de/*.md und sdk/README.de.md mit tools/mkdocs.py (make docs).

# Inhalt

V-OS ist ein grafisches Betriebssystem für den Amstrad CPC 6128, von Grund auf in Z80-Assembler geschrieben. Es bietet:

- eine Menüleiste, verschiebbare Fenster, Dialoge und Schaltflächen, bedienbar mit der Tastatur oder einer AMX-Maus
- drei Ansichten für Dateien und Programme (Schreibtisch, Programme, Commander) und die Einstellungen (Optionen)
- sechs Sprachen als Plugins: Englisch, Griechisch, Französisch, Spanisch, Deutsch und Italienisch
- Programme, die nebeneinander laufen und dort weitermachen, wo sie aufgehört haben, ein Zubehörprogramm und Programme im Hintergrund
- Studio, einen Block-Editor im Stil von Scratch, der auf dem CPC selbst läuft
- eine API mit 42 Aufrufen und ein SDK für Programme in Assembler

Dieses Handbuch gibt es auch auf Griechisch, Englisch, Französisch, Spanisch und Italienisch.


- 1. Erste Schritte
- 2. Die Ansichten
- 3. Programme
- 4. Sprachen
- 5. Studio: Programme mit Blöcken
- 6. Blöcke auf dem PC: der Web-Editor und `vbc.py`
- 7. Das V-OS-SDK: Programme in Assembler
- 8. V-OS entwickeln

# 1. Erste Schritte

## Was du brauchst

- Einen Amstrad CPC 6128 oder einen Emulator dafür. V-OS wird mit Caprice32 getestet.
- Die V-OS-Diskette in Laufwerk A:, also die `build/vos.dsk`, die `make` erzeugt, und die Diskette der Programme, `build/apps.dsk`, in B:.
- Optional:
  - eine AMX-Maus
  - ein zweites Laufwerk (B:)
  - eine Speichererweiterung (z. B. 512K), für mehr Programme gleichzeitig

## Starten

Aus BASIC:

```
RUN"VOS
```

V-OS lädt den Kernel, schaltet die ROMs ab und übernimmt die ganze Maschine. Eine Speichererweiterung erkennt es von selbst.

- **Erste Ansicht:** der Schreibtisch. Du kannst das in den Optionen ändern, in der Zeile START-ANSICHT.
- **Sprache:** Englisch. Liegt eine `SETUP.CFG` auf der Diskette, startet es mit der Sprache und den Einstellungen, die dort gespeichert sind.
- **Beenden:** DATEI > ZURÜCK ZU BASIC (CONTROL+Q in den Ansichten). Es fragt nach einer Bestätigung und startet den CPC neu.

## Der Bildschirm

V-OS nutzt den ganzen Bildschirm (Overscan): 384×272 Pixel in Modus 1 (4 Farben) oder 192×272 in Modus 0 (16 Farben).

- **Menüleiste** (oben): die Menüs der Ansicht oder des Programms, meist DATEI oder das Menü des Programms, ANSICHT und HILFE. Rechts steht die Tastatursprache (EN, EΛ, ...).
- **Unterste Zeile:** in jeder Ansicht die Tasten, die dort funktionieren.

## Tasten, die überall funktionieren

| Taste | Was sie tut |
|---|---|
| F1 | Schreibtisch |
| F2 | Programme |
| F3 | Commander |
| F4 | Optionen |
| F5 | öffnet und schließt das Zubehörprogramm (ZUBEHÖR) |
| F6 | die laufenden Programme, um eines auszuwählen |
| ESC | öffnet das erste Menü. In den Menüs wechseln ←/→ das Menü, ↑/↓ wählen einen Eintrag, RETURN führt ihn aus, ESC schließt |

| | |
|---|---|
| CONTROL+Buchstabe | die Kurzbefehle der Menüs (z. B. ^Q, ^L) |
| CONTROL+L | Sprache wechseln (ANSICHT > SPRACHE WECHSELN) |
| CONTROL+K | der Dialog mit den Tasten (HILFE > TASTEN) |
| CONTROL+SHIFT | wechselt die Tastatur: Englisch, die des Pakets und, mit TASTATUREN: ALLE SECHS in Setup, die der anderen Sprachen |
| TAB, SHIFT+TAB | bewegen den Fokus zwischen den Schaltflächen. RETURN, ENTER oder SPACE drücken die Schaltfläche, die den Fokus hat |

F1–F6 sind die Tasten f1–f6 auf dem Ziffernblock des CPC.

## Der Zeiger und die Maus

- **Mit einer AMX-Maus:** Der Zeiger bewegt sich mit der Maus, und ihre Taste klickt. Die Maus wird in den Optionen eingeschaltet und eingestellt (AMX-MAUS, AMX-LESEART).
- **Ohne Maus:** Der Zeiger bewegt sich mit den Pfeiltasten, und COPY klickt. In Ansichten, die die Pfeiltasten zum Auswählen nutzen (Schreibtisch, Programme, Commander), wählen die Pfeile Dateien und Programme aus, statt den Zeiger zu bewegen.

Das meiste funktioniert mit beidem: Ein Klick auf eine Datei oder ein Programm wählt es aus, ein zweiter Klick öffnet es.

## Fenster und Dialoge

- **Verschieben:** Jedes Fenster lässt sich an seiner Titelleiste verschieben: die Dialoge, das Fenster DATEIEN des Schreibtischs, das Zubehörprogramm und die Fenster von Blockprogrammen. Solange du die Taste hältst, wird ein gestrichelter Umriss gezeigt, und das Fenster wird dort neu gezeichnet, wo du loslässt.
- **Dialoge:** warten auf eine Antwort. TAB und ←/→ wechseln die Schaltfläche, RETURN drückt sie, und ESC wählt die letzte (meist NEIN oder ABBRUCH).
- **Texteingabe** (z. B. beim Umbenennen): Du tippst am Ende, DEL löscht, RETURN übernimmt und ESC bricht ab. Du kannst auch in der Sprache des Sprachpakets tippen.

## Disketten und Dateien

- **Format:** V-OS liest und schreibt AMSDOS-Disketten (Format DATA oder SYSTEM) in A: und B:. Auch BASIC kann seine Dateien lesen.
- **Namen:** 8 Zeichen plus 3 für den Typ (`NAME.TYP`).

Typen, denen du begegnen wirst:

| Typ | Was es ist |
|---|---|
| `.BIN` | ein V-OS-Programm (oder eine Binärdatei) |
| `.VBN` | ein V-OS-Programm, das mit Blöcken erstellt wurde |
| `.VBS` | ein Studio-Programm (sein Quelltext) |
| `.OVL`, `.MSG`, `.LNG` | Teile von V-OS selbst: Ansichten, Texte, Sprachen |
| `.SYS` | Systemdateien (z. B. `VBRUN.SYS`, der Player von Studio) |
| `SETUP.CFG` | die Einstellungen |

Die Einstellungen, Sprachen und Ansichten werden von A: gelesen, deshalb muss die V-OS-Diskette dort bleiben.

## Fehlermeldungen

Diskettenfehler werden in einem Dialog angezeigt: NICHT BEREIT, SCHREIBGESCHÜTZT, DATEI NICHT GEFUNDEN, DISKETTE VOLL, VERZEICHNIS VOLL, NAME SCHON VERGEBEN, UNGÜLTIGER NAME und andere.

Zwei weitere betreffen Programme:

- **KEIN V-OS-PROGRAMM:** Die Datei hat nicht den Kopf eines Programms.
- **ERST EIN ANDERES PROGRAMM SCHLIESSEN:** Es ist kein Platz frei. Siehe Kapitel 3.

# 2. Die Ansichten

V-OS hat vier „Ansichten“: Arten, die Dateien und Programme zu sehen, und die Einstellungen. Du wechselst die Ansicht mit F1–F4 oder über das Menü ANSICHT. Jede Ansicht merkt sich, welche Datei ausgewählt war, und die nächste wählt dieselbe Datei aus.

## Die gemeinsamen Menüs

Die letzten beiden Menüs sind in allen Ansichten und in Programmen gleich:

| **ANSICHT** | | **HILFE** | |
|---|---|---|---|
| SCHREIBTISCH | F1 | TASTEN | ^K |
| PROGRAMME | F2 | ÜBER V-OS | |
| COMMANDER | F3 | | |
| OPTIONEN | F4 | | |
| PROGRAMME (laufende) | F6 | | |
| SPRACHE WECHSELN | ^L | | |

HILFE > ÜBER V-OS zeigt die Version (0.15 Beta; jeder Meilenstein fügt 0.01 hinzu), das Datum des Builds, den Hersteller (Revive8bit, 2026) und den Architekten (Vasper).

## Schreibtisch, F1

Modus 1. Ein Fenster DATEIEN zeigt die Dateien eines Laufwerks als Symbole, vier pro Reihe. Der Typ steht im Symbol und der Name darunter, und eine `.VBN` hat daneben ein „Play“-Zeichen. Rechts wählen zwei Symbole A: oder B:. Die Titelleiste zeigt das Diskettenformat und die Anzahl der Dateien, und unten wird der freie Platz angezeigt.

| **Taste** | **Was sie tut** |
|---|---|
| Pfeile | eine Datei auswählen (↑/↓ eine Reihe) |
| TAB | das andere Laufwerk |
| RETURN, ENTER | öffnen: ein Programm startet, eine Datei bekannten Typs öffnet sich in ihrem Programm, andere zeigen ihre Info |
| ^C | auf das andere Laufwerk kopieren |
| ^D | löschen (fragt vorher) |
| ^N | umbenennen |
| ^R | die Disketten neu lesen |
| Maus | Klick auf eine Datei: auswählen, noch einmal: öffnen. Klick auf ein Laufwerk: öffnet es |

- **Menü DATEI:** ÖFFNEN, DATEI-INFO (^I), AUF ANDERES LAUFWERK, LÖSCHEN, UMBENENNEN, DISKETTEN NEU LESEN, ZURÜCK ZU BASIC.
- **Das Fenster DATEIEN** lässt sich an seinem Titel verschieben und bleibt dort, auch wenn du die Ansicht wechselst.
- **Öffnen nach Typ** (auch im Commander): `.TXT`-Dateien öffnen sich in NOTEPAD, `.MD` in MARKDOWN, `.PIC` und `.LYR` in PAINT, `.VDB` in DATABASE, `.VSH` in SHEET, `.MUS` in MUSIC, das sie gleich abspielt. Das Programm wird auf dem Laufwerk der Datei gesucht, dann auf A:, dann auf B:. Wird es nicht gefunden, erscheint „KEIN PROGRAMM FÜR DIESE DATEI“. Hat das Programm ungespeicherte Änderungen, fragt es zuerst.

## Programme, F2

Modus 0, 16 Farben. Zeigt nur die Programme eines Laufwerks (`.BIN`, `.VBN`, `.BAS`), zuerst A:, als Kacheln in einem 3×3-Raster, mit den weiteren Seiten darunter. Die Seitenzeile beginnt mit dem Laufwerk (`A:`, `B:`). Jedes Programm, das ein Symbol hat, zeigt es auf seiner Kachel. Die ausgewählte Kachel ist weiß.

| Taste | Was sie tut |
|---|---|
| Pfeile | auswählen (↑/↓ eine Reihe. Über den Rand hinaus die nächste oder vorige Seite) |
| RETURN, ENTER | startet das Programm. Läuft es bereits, macht es dort weiter, wo es war |
| ^A | lädt es als Zubehörprogramm (ALS ZUBEHÖR). F5 öffnet es dann |
| ^R | die Diskette neu lesen |
| TAB | das andere Laufwerk (ANDERES LAUFWERK). Ein Programm, das von B: gestartet wurde, kehrt zu B: zurück |
| Maus | Klick: auswählen, noch einmal: starten |

**Menü DATEI:** STARTEN, ALS ZUBEHÖR, ANDERES LAUFWERK, DISKETTEN NEU LESEN, ZURÜCK ZU BASIC.

## Commander, F3

Modus 1, zwei Felder mit Dateilisten: A: links, B: rechts. Jedes Feld zeigt in seinem Titel das Format und die Anzahl der Dateien, und unten den freien Platz. Das aktive Feld hat einen dunklen Titel und eine Auswahl.

| Taste | Was sie tut |
|---|---|
| ↑/↓ | auswählen (mit SHIFT: eine Seite) |
| TAB, ←/→ | das andere Feld |
| RETURN, ENTER | öffnen, wie auf dem Schreibtisch. Die Info (Größe, Typ, Ladeadresse, die ersten Bytes) mit ^I |
| ^C | in das andere Feld kopieren, in beliebiger Größe. Gibt es eine Datei mit demselben Namen, fragt es vorher |
| ^D | löschen |
| ^N | umbenennen |
| ^R | beide Disketten neu lesen |
| Maus | Klick: auswählen, noch einmal: öffnen |

**Menü DATEI:** ÖFFNEN, DATEI-INFO (^I), INS ANDERE FELD KOPIEREN, LÖSCHEN, UMBENENNEN, DISKETTEN NEU LESEN, FORMATIEREN (DATA), ZURÜCK ZU BASIC.

> FORMATIEREN löscht alle Dateien auf der Diskette des aktiven Feldes. Es fragt vorher.

## Optionen, F4

Modus 1. Eine Zeile für jede Einstellung. Änderungen wirken sofort. Beim Speichern (RETURN oder ^S) werden sie in `A:SETUP.CFG` geschrieben und gelten auch beim nächsten Start.

| Einstellung | Werte |
|---|---|
| START-ANSICHT | SCHREIBTISCH, PROGRAMME, COMMANDER |
| SPRACHE | ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO |
| VERZÖGERUNG | wie lange du eine Taste hältst, bevor sie sich wiederholt: KURZ, MITTEL, LANG |

| | |
|---|---|
| WIEDERHOLRATE | LANGSAM, MITTEL, SCHNELL |
| AMX-MAUS | AN, AUS |
| AMX-LESEART | IMPULSE oder PEGEL: wie die Maus gelesen wird. Bewegt sich der Zeiger nicht richtig, versuche die andere |
| HINTERGRUND | BLAU, SCHWARZ, GRÜN, ROT |
| TASTATUREN | EN + SPRACHE, ALLE SECHS (die CONTROL+SHIFT durchgeht) |

| Taste | Was sie tut |
|---|---|
| ↑/↓ | eine Einstellung auswählen |
| ←/→ | den Wert ändern (auch ein Klick auf die Zeile) |
| RETURN, ^S | speichern |

Wenn du die SPRACHE änderst, wird das Sprachpaket sofort von A: geladen. Siehe Kapitel 4.

# 3. Programme

## Starten und Beenden

Ein Programm startet auf eine dieser Arten:

- aus der Ansicht Programme: auswählen und RETURN drücken, oder zweimal klicken
- vom Schreibtisch oder aus dem Commander: DATEI > STARTEN
- aus einem anderen Programm, etwa Studio, das alles startet, was du erstellst

Jedes Programm hat sein eigenes Menü neben den gemeinsamen Menüs ANSICHT und HILFE. Es endet über sein Menü (meist BACK oder STOP, CONTROL+Q) und kehrt zu der Ansicht zurück, aus der es gestartet wurde.

## Mehrere Programme gleichzeitig

Es gibt zwei Arten von V-OS-Programmen:

- **Kleine, mit einer Relokationstabelle:** DEMO, MIXED und CLOCK, die Beispiele des Web-Editors (PONG, CATCH, STARCAT, COUNTER) und alles, was das SDK erzeugt. Sie teilen sich 12K Speicher, und so viele, wie hineinpassen, laufen zusammen.
- **Programme, die den ganzen Speicher brauchen:** STUDIO und alles, was Studio erzeugt (CALC, MINES, FLAP, TICKER und deine eigenen). Ohne Speichererweiterung laufen sie nur allein. Läuft schon eines, erscheint „ERST EIN ANDERES PROGRAMM SCHLIESSEN“.

Wenn du zu einer Ansicht wechselst (F1–F4), während ein Programm läuft:

- Ein kleines Programm bleibt im Speicher. Du holst es in der Ansicht Programme (RETURN darauf) oder mit F6 wieder hervor, und es macht genau dort weiter, wo es war.
- Ein Programm, das den ganzen Speicher hat, endet, außer es gibt eine Speichererweiterung.

**F6** (ANSICHT > PROGRAMME) zeigt die Liste der laufenden Programme. Du wählst eines mit den Pfeilen und RETURN. Läuft keines, erscheint „KEIN PROGRAMM LÄUFT“.

**Mit einer Speichererweiterung** (z. B. 512K, die V-OS beim Start von selbst findet) bekommt jedes Programm, das den ganzen Speicher braucht, eine eigene Bank. So können STUDIO, CALC, DEMO und PONG alle zusammen laufen. Seit M43 bekommt jedes Programm auch eine eigene Datenbank von 16K, wenn es sie braucht: was es dort hält, geht nicht verloren, wenn andere Programme oder Ansichten laufen.

## Im Hintergrund

Ein Programm hinter einem anderen oder hinter einer Ansicht **friert** normalerweise **ein** und macht weiter, wenn es wieder nach vorne kommt.

Manche Programme verlangen, auch im Hintergrund **weiterzulaufen**, z. B. TICKER, das die Zeit zählt. Solange sie dort sind, zeichnen sie nicht und spielen keinen Ton. Wenn sie wieder nach vorne kommen, haben sie auch die Zeit mitgezählt, in der sie weg waren.

In Studio wählst du das für dein Programm mit STUDIO > IN BACKGROUND.

## Das Zubehörprogramm (ZUBEHÖR)

Ein kleines Programm wie CLOCK kann sich in einem Fenster **über allem** öffnen: über einer Ansicht oder einem Programm.

1. Wähle in der Ansicht Programme CLOCK aus und dann DATEI > ALS ZUBEHÖR (^A).
2. Ab dann öffnet und schließt **F5** es, wo du auch bist. Auch ESC schließt es.

Solange es offen ist, bekommt es die Tasten und Klicks. Sein Fenster lässt sich an seinem Titel verschieben. Wenn es sich schließt, wird neu gezeichnet, was darunter war.

## Die Programme auf der Diskette

Eine CPC-Diskette fasst 180K, darum kommt V-OS auf zwei. `vos.dsk` (A:) hat das System, DEMO, MIXED, STUDIO und CLOCK und die `.VBS`-Quellen der VB-Programme, die Studio von A: öffnet. `apps.dsk` (B:) hat die anderen, von PONG bis WEBGET: im Launcher zeigt TAB das Laufwerk B:. Manche Programme haben eine eigene Diskette, wie die Tabelle sagt.

| Programm | Was es ist |
|---|---|
| **STUDIO** | der Block-Editor: Du erstellst Programme auf dem CPC (Kapitel 5) |
| **DEMO** | das erste V-OS-Programm: ein Quadrat, das sich von selbst bewegt. Die Pfeile steuern es, und SPACE ändert seine Farbe |
| **MIXED** | der „gemischte Bildschirm“: ein Streifen mit 16 Farben (Modus 0) zwischen Streifen in Modus 1. Ein Klick zeigt die Koordinaten |
| **CLOCK** | die Zeit seit dem Start von V-OS. Läuft auch als Zubehörprogramm |
| **CALC** | ein Taschenrechner in einem Fenster mit Schaltflächen, erstellt in Studio (es gibt auch `CALC.VBS`) |
| **MINES** | ein Minesweeper mit 8×6 Feldern und 8 Minen, erstellt in Studio. Ein Klick deckt ein Feld auf, und NEW startet ein neues Spiel |
| **BRICKS** | Mauerbrechen: ein Schläger mit ←/→, 32 Steine, 3 Leben, Level mit einem immer schnelleren Ball. Bei GAME OVER startet SPACE ein neues Spiel |
| **VROUVES** | Tower Defence, nach dem CPC-Spiel LET'S GO FOR VROUVES: 7 Level, vom Kafenio bis zum Friedhof, mit je 10 Wellen. Es steht auf seiner eigenen Diskette, `vrouves.dsk`: in B:, und TAB im Launcher. Die Pfeile bewegen den Cursor, 1–6 wählt einen Verteidiger, SPACE baut, S verkauft, N holt die nächste Welle, P pausiert. Punkte für jeden Eindringling und für die Dorfbewohner, die am Ende eines Levels übrig sind. Die Diskette speichert `VROUVES.SAV`: CONTINUE beginnt mit dem erreichten Level, HIGH SCORES zeigt die besten fünf mit Namen. Mit dem SDK geschrieben (`myapps/vrouves`) |
| **NOTEPAD** | ein Texteditor in Modus 1, 48×22, für Dateien bis 4K, oder bis 16K mit einer Speichererweiterung: dann hält es etwa 2,5K um den Cursor in seinem Speicher und den Rest in seiner eigenen Datenbank, und SELECT ALL nimmt dieses Stück. NEW, OPEN (nur `.TXT`-Dateien), SAVE, SAVE AS, OTHER DRIVE. Pfeile, DEL, CLR und RETURN; ein Klick setzt den Cursor. Die untere Zeile zeigt Zeile und Spalte. Menü EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT und die Pfeile markieren. Die Zwischenablage (bis 1K) teilen sich alle Programme (`myapps/notepad`) |
| **PAINT** | Malen in Modus 0 mit 16 Farben: ein Bild mit 128×128 (`.PIC`), oder 64×64 mit 2×-Zoom in 3 Ebenen (`.LYR`, NEW LAYERS). Werkzeuge: P Stift, B Pinsel, L Linie, X Rechteck, F Füllen; TAB wechselt die Farbe. Pfeile und COPY malen ohne Maus. Ebenen: 1–3 wählt eine, H blendet sie aus oder ein, U und D schieben sie nach oben und unten, E ist der Radierer. EXPORT .SCR: das Bild (die Ebenen, wie sie zu sehen sind) als CPC-Bildschirm, eine 16K-Datei `NAME.SCR` (Modus 0, in der Mitte, mit der Palette), für BASIC oder andere Programme. In BASIC: `MODE 0:LOAD"NAME.SCR",&C000:FOR I=0 TO 15:INK I,PEEK(&C7D0+I):NEXT:BORDER PEEK(&C7E0)` (`myapps/paint`) |

| | |
|---|---|
| **MUSIC** | spielt Songs von Arkos Tracker 3 (`.MUS`, mit `make songs` erzeugt), mit drei Lautstärkebalken. Es steht auf seiner eigenen Diskette, `music.dsk`, in B:. OPEN, SPACE spielt und stoppt. Die Musik läuft hinter anderen Ansichten und Programmen weiter. Studios MUSIC-Block startet es mit einem Lied hinter einem Programm (`myapps/music`) |
| **MARKDOWN** | ein Editor für Markdown-Dateien (`.MD`, bis 6K), wie NOTEPAD. ^P zeigt die Seite: `#`-Überschriften, **fett**, *kursiv*, `Code`, Listen, Zitate, Linien, Codeblöcke und unterstrichene Links, ohne die Zeichen. Aufeinanderfolgende Zeilen werden ein Absatz (zwei Leerzeichen am Ende brechen die Zeile um). Die Pfeile blättern die Seite, RETURN oder ^P führen zurück zum Text. Es steht auf seiner eigenen Diskette, `markdown.dsk`, mit einer `README.MD`. Menü EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT und die Pfeile markieren. Die Zwischenablage (bis 1K) teilen sich alle Programme (`myapps/markdown`) |
| **SYSINFO** | zeigt die Versionen von V-OS und der API, den Speicher (den ganzen und den für Programme freien) und den freien Platz auf den Disketten in A: und B: (^R erneut). Ein Beispiel für `VOS_EXT` (`myapps/sysinfo`) |
| **WEBGET** | lädt Dateien aus dem Web mit der M4-Karte: GET A FILE (^G) fragt nach einer Adresse (HTTP, z. B. `example.com/index.html`) und einem Namen, und die Datei (bis 16K) wird auf dem angezeigten Laufwerk gespeichert (OTHER DRIVE). Es zeigt auch den Netzwerkstatus. Ohne M4-Karte: „NO M4 BOARD“ (`myapps/webget`) |
| **READER** | liest Markdown-Seiten (`.MD`, bis 6K) von der Diskette oder mit der M4-Karte aus dem Web, so dargestellt wie die Vorschau von MARKDOWN. TAB wählt den nächsten Link (invertiert), RETURN öffnet ihn, DEL geht zur vorigen Seite zurück (es merkt sich 4). Ein Link ohne `http://` öffnet sich neben der Seite, auf der Diskette oder auf derselben Website. OPEN A FILE und OPEN AN ADDRESS (^U, HTTP, z. B. `example.com/README.md`). `https://`-Adressen lassen sich nicht öffnen. Es ist auf einer eigenen Diskette, `reader.dsk`, mit `INDEX.MD` und `ABOUT.MD` (`myapps/reader`) |
| **STREETS** | ein Straßenverzeichnis in einer V-OS-Datenbank: tippe die ersten Buchstaben einer Straße und sieh die Straßen, die so beginnen, mit Gebiet und Postleitzahl; DEL löscht einen Buchstaben. ^A fügt eine Straße hinzu, ^D löscht die erste der Liste. Die Straßen sind in `STREETS.VDB` auf seiner Diskette, `streets.dsk`, in B:; beim ersten Mal legt es sie mit Straßen Athens an. Ein Beispiel der Datenbank-API (`myapps/streets`) Beim Suchen zählen Groß- und Kleinschreibung und Akzente nicht. |
| **DATABASE** | Datenbanken, wie ein kleines Access: ^N eine neue Datenbank (`NAME.VDB`), ^O eine öffnen, ^T eine neue Tabelle (ihr Name und Felder: ein Name, TEXT bis 40 Zeichen oder ZAHL 0–65535, und ob sie nach dem ersten sortiert wird). Die Liste zeigt einen Datensatz pro Zeile und die Felder in Spalten; die Pfeile wählen und blättern, TAB geht zur nächsten Tabelle, und hat die Tabelle einen Schlüssel, springt das Getippte zu den Datensätzen, die so beginnen (ohne Groß-/Kleinschreibung und Akzente). RETURN öffnet das Formular: ein Feld pro Zeile, RETURN ändert es, ^S speichert, ^B zurück zur Liste. ^A ein neuer Datensatz, ^D löschen, ^W das andere Laufwerk. Es öffnet auch die `.VDB` von STREETS (`myapps/database`) |
| **SHEET** | eine Tabellenkalkulation: 16 Spalten (A–P) und 64 Zeilen, 5 Spalten und 19 Zeilen auf dem Bildschirm. Eine Zelle enthält, bis zu 40 Zeichen, eine Zahl (`12`, `-3.5`; das Dezimalzeichen ist ein Punkt), Text oder eine Formel nach `=`: `+ - * /`, Klammern, Zellen (`B3`), SUM, AVG, MIN, MAX und COUNT von Zellen und Bereichen (`=SUM(B2:B9)`), ABS, INT, ROUND(x, 0–2). Zahlen haben zwei Dezimalstellen, bis ±21474836.46. Die Pfeile bewegen den Cursor, RETURN ändert die Zelle, Tippen beginnt einen neuen Text, DEL (oder CLR) leert sie; nach jeder Änderung werden alle Formeln neu berechnet. Die erste Zeile zeigt den Text der Zelle. Eine Zelle mit Fehler zeigt `#ERR`, eine zu breite Zahl `########`. ^N neu, ^O öffnen, ^S speichern, ^A speichern unter (`NAME.VSH`, eine Textdatei, eine Zeile je Zelle: `B3 =B1*2`), ^W das andere Laufwerk. Die Formeln berechnet `CALC.SYS` auf A:, das jedes Programm benutzen kann (`VX_CALC`). Es ist auf einer eigenen Diskette, `sheet.dsk`, mit dem Beispiel `BUDGET.VSH` (`myapps/sheet`) |
| **PONG** | ein Ball und ein Schläger mit den Pfeilen, Punkte und Leben |
| **CATCH** | fange die fallenden Sterne |

| | |
|---|---|
| **STARCAT** | eine Katze mit Kostümen, die läuft, ein Stern, eine Melodie aus einer Liste |
| **COUNTER** | ein Zähler mit den Schaltflächen +1, −1 und 0, in Modus 1 |
| **FLAP** | ein Vogel, der mit den Flügeln schlägt (Kostüme), und ein Stern, der jede Sekunde das Kostüm wechselt |
| **TICKER** | zählt Sekunden und läuft im Hintergrund weiter |

- Blockprogramme sind `.VBN`-Dateien. Sie haben ein Menü PROGRAM > STOP (CONTROL+Q).
- Die, die auch eine `.VBS` haben (CALC, MINES, FLAP, TICKER), lassen sich in Studio öffnen, damit du sie ansehen oder ändern kannst.
- `VBRUN.SYS` ist kein Programm: Es ist der „Player“, den Studio braucht.

# 4. Sprachen

V-OS spricht sechs Sprachen. Englisch ist eingebaut. Die anderen fünf sind **Sprachpakete**, Dateien auf der Diskette, die drei Dinge mitbringen: alle Systemtexte, die Tastatur und die Buchstaben (Schrift) der Sprache.

| Sprache | Datei | Tastatur |
|---|---|---|
| English | eingebaut (`ENGLISH.MSG`) | die englische des CPC |
| Ελληνικά | `GREEK.LNG` | Griechisch, mit Akzenten |
| Français | `FRENCH.LNG` | AZERTY, Ziffern mit SHIFT, Tottasten ^ und ¨ |
| Español | `SPANISH.LNG` | Ñ auf der Taste `:`, Tottasten ´ ¨ ` ^, und ¡ ¿ |
| Deutsch | `GERMAN.LNG` | QWERTZ, Ü Ö Ä ß, Tottasten ´ und ` |
| Italiano | `ITALIAN.LNG` | è é ò à ù ì auf eigenen Tasten |

## Die Sprache wechseln

- **Dauerhaft:** In den Optionen (F4) geht die Zeile SPRACHE der Reihe nach durch alle Sprachen. Jede wird sofort von A: geladen, und der ganze Bildschirm wird in ihr neu gezeichnet. Speichern (^S) behält sie auch für den nächsten Start.
- **Schnell:** CONTROL+L (ANSICHT > SPRACHE WECHSELN) schaltet zwischen Englisch und der Sprache des geladenen Pakets um.
- **Nur die Tastatur:** CONTROL+SHIFT wechselt die Tastatur. Die Menüleiste zeigt rechts, welche aktiv ist (EN, EΛ, FR, ...). Mit TASTATUREN in Setup auf EN + SPRACHE geht es zwischen der englischen und der des Pakets; auf ALLE SECHS durch alle: EN, die des Pakets, die vier anderen und wieder EN. So kannst du Französisch mit der französischen Tastatur tippen, während das System auf Griechisch ist. Jedes Paket enthält die Tastaturen aller.

Die Zeichen `@ # [ ] { } \ |` gibt es auf den Tastaturen der Pakete nicht, weil der CPC keine AltGr-Taste hat. Tippe sie mit der englischen Tastatur.

## Die Texte der Programme

Programme haben ihre eigenen Texte in **zwei Sprachen**: Englisch und, optional, einer weiteren (heute Griechisch). Ein Programm erscheint nur dann in seiner zweiten Sprache, wenn das Paket in dieser Sprache ist. Sonst erscheint es auf Englisch. Mit Französisch erscheint Studio also auf Englisch.

## Ein Alphabet für alle

Alle Sprachen schreiben mit demselben Zeichensatz, dem **V-OS-Zeichensatz**, und jedes Paket hat alle seine Buchstaben. So kann ein Text oder eine Datenbank Griechisch, Französisch, Deutsch, Spanisch und Italienisch mischen, und alles erscheint richtig, welche Sprache auch aktiv ist: in Setup zeigt LANGUAGE ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL mit ihren eigenen Buchstaben.

Die Codes 128 bis 255 sind die griechischen von ISO 8859-7, unverändert, und die Codes, die es frei lässt, tragen À Á Â Ä Ç È É Ê Ë Ì Í Î Ï Ñ Ò Ó Ô Ö Ù Ú Û Ü, ihre Kleinbuchstaben, ß und ¡ ¿. Der Akzent ´ wird als griechischer Tonos ( ΄ ) geschrieben und µ als µ. Es gibt kein ÿ, Œ oder œ. Die Tabelle steht in `tools/voscs.py`. Eine griechische Textdatei vom PC (ISO 8859-7) wird so gelesen, wie sie ist; eine französische oder deutsche in Latin-1 muss umgewandelt werden.

Mit der englischen Tastatur gibt SHIFT und ^ das £.

Dateien in **UTF-8**, vom PC oder aus dem Web, öffnen NOTEPAD und READER und wandeln sie in den V-OS-Zeichensatz um; was er nicht hat (etwa Chinesisch), wird `?`. Auf dem PC wandelt `tools/voscv.py` Dateien in V-OS und zurück um (UTF-8, Latin-1 und andere) und sagt, was eine Datei ist (`check`).

## Eine neue Sprache erstellen

Die Quellen der Sprachen liegen in `lang/<code>/`:

- `messages.txt`: die Systemtexte. Die Grundliste legt `lang/en/messages.txt` fest, und was in einem Paket fehlt, erscheint auf Englisch.
- `keymap.txt`: die Tasten, die sich von der englischen Tastatur unterscheiden.
- `pack.txt`: der Code, der Name, der Zeichensatz und die Schrift.

`tools/mklng.py` macht daraus eine `.LNG`, und das Makefile legt sie auf die Diskette. Alle Pakete haben dieselbe Schrift, `lang/vos/font.txt`, und `charset vos`.

# 5. Studio: Programme mit Blöcken

STUDIO ist ein Block-Editor im Stil von Scratch, der auf dem CPC selbst läuft.

- Du baust Sprites und Skripte aus Blöcken.
- Das Programm läuft sofort mit CONTROL+R.
- Es wird ein normales V-OS-Programm (`.VBN`), das auch aus der Ansicht Programme startet.

## Der Bildschirm

Studio nutzt einen „geteilten" Bildschirm mit zwei Zonen:

- **Oben, in Modus 0 (16 Farben):**
  - **links die Bühne** in halber Größe
  - **rechts das Feld SPRITES:** die Sprites mit ihrer Farbe und Position, die Größe des ausgewählten (W, H), der Hintergrund und der Modus des Programms (z. B. `STAGE 2 M1`) sowie der Dateiname
- **Unten, in Modus 1:**
  - **links die Palette** mit den Blöcken einer Kategorie (`< EVENTS >`)
  - **rechts das Skript**, eine Zeile pro Block
  - **ganz unten die Statuszeile**, mit den Tasten, die dort funktionieren, wo du gerade bist

**TAB** geht vom Skript zur Palette, dann zur Bühne und zurück zum Skript.

## Menüs

| STUDIO | | EDIT |
|---|---|---|
| NEW | ^N | DELETE LINE |
| OPEN (`.VBS`) | ^O | + SPRITE (bis zu 6) |
| SAVE | ^S | − SPRITE |
| SAVE AS | | BACKGROUND (die Farbe der Bühne) |
| RUN | ^R | SCREEN MODE (0 oder 1) |
| IN BACKGROUND | | |
| BACK | ^Q | |

- **RUN (^R):** speichert `NAME.VBS`, schreibt `NAME.VBN` und startet es. Das Programm endet über sein eigenes Menü (PROGRAM > STOP, ^Q), und dann startet Studio wieder mit demselben Programm geöffnet.
- **IN BACKGROUND:** fragt, ob das Programm weiterläuft, wenn es hinter einem anderen oder hinter einer Ansicht liegt, oder pausiert, was die Voreinstellung ist.
- **SCREEN MODE:**
  - **Modus 0:** 16 Farben, 192 Pixel breit.
  - **Modus 1:** 4 Farben (Stifte 0–3), 384 Pixel breit, und mehr Text.

## Die Palette

←/→ wechseln die Kategorie, ↑/↓ wählen einen Block, und RETURN fügt ihn unter der Skriptzeile ein. Ein Klick auf einen Block der Palette tut dasselbe.

Ein Block, der sich öffnet (FOREVER, REPEAT, IF), kommt zusammen mit seinem END hinein und wird zusammen mit ihm gelöscht. Ein Block mit Text fragt sofort nach dem Text.

## Das Skript

| Taste | Was sie tut |
|---|---|
| ↑/↓ | Zeile |
| ←/→ | Feld der Zeile (die Felder sind die Werte im Block) |
| RETURN | ohne ausgewähltes Feld: fügt den Block der Palette ein. Auf einem Textfeld: den Text ändern. Auf einem Tastenfeld: wartet, bis du die Taste drückst |
| CLR | löscht die Zeile |
| DEL | löscht die letzte Ziffer einer Zahl |
| Maus | Klick auf ein Feld: wählt es aus. Zweiter Klick: ändert seine Art |

Das Programm ist eine Liste von Zeilen:

- Jedes `WHEN …` beginnt ein Skript, für ein Sprite oder für die Bühne. Das Skript reicht bis zum nächsten `WHEN`.
- Die Einrückung ergibt sich von selbst aus den Blöcken, die sich öffnen und schließen.
- Zeilen vor dem ersten `WHEN` werden ignoriert.
- Was offen bleibt, wird am Ende des Skripts geschlossen.

### Was in ein Feld kommt

Die Statuszeile sagt immer, was das Feld annimmt.

**Wertfeld:**

| Taste | Wert |
|---|---|
| 0–9, − | Zahl, eine ganze Zahl von −32768 bis 32767 |
| A–H | Variable. Derselbe Buchstabe noch einmal: A1, A2 … A15, und zurück zu A |
| X, Y | die Position des Sprites des Skripts |
| R und Ziffern | RANDOM: eine Zahl von 1 bis n |
| K und eine Taste | KEY: 1, solange sie gedrückt ist, sonst 0 |
| T und eine Ziffer | TOUCHING: 1, wenn das Sprite das Sprite n berührt |
| Z | AT EDGE: 1, wenn das Sprite am Rand der Bühne ist |
| P | BUTTON: die Nummer der zuletzt gedrückten Schaltfläche |
| U | COSTUME: die Nummer des Kostüms des Sprites (0: kein Kostüm) |

**Andere Arten von Feldern:**

- **Variable:** A–H, und derselbe Buchstabe noch einmal für A1…A15.
- **Vergleich:** `<`, `=`, `>`. SPACE ergibt „keiner“, und dann prüft IF nur, ob der erste Wert nicht 0 ist.
- **Sprite** (in den `WHEN`-Blöcken): 0 die Bühne, 1–6 ein Sprite.
- **Taste** (in `WHEN KEY`): RETURN und dann die Taste.
- **Text:** RETURN, und du tippst bis zu 14 Zeichen.

- **Variablengruppe** (SAVE/LOAD VARS): A–H geht zur nächsten Achtergruppe (A-H, A1-H1 … A15-H15), und SPACE ergibt ALL.

Die Buchstaben werden als Tasten gelesen, deshalb funktionieren sie auch mit der griechischen Tastatur.

## Die Bühne und die Sprites

| Taste | Was sie tut |
|---|---|
| Pfeile | bewegen das ausgewählte Sprite |
| 1–6 | ein Sprite auswählen |
| C | nächste Farbe |
| W, H | Breite, Höhe |
| K | welches Kostüm es trägt (keines, 1, 2, …) |
| E | sein Kostüm zeichnen (ein neues, wenn es keines hat) |
| Klick auf die Bühne | setzt das Sprite dorthin |

Ein Sprite ohne Kostüm ist ein farbiges Rechteck.

### Kostüme

**E** öffnet den Kostüm-Editor an der Stelle der Bühne. Er hat ein 16×16-Raster mit den 16 Stiften darunter.

| Taste | Was sie tut |
|---|---|
| Pfeile | Zelle |
| SPACE, RETURN | malt die Zelle |
| 0–9, A–F | Stift |
| . | transparent |
| E, TAB | fertig |
| Klick | auf eine Zelle: malt. Auf die Reihe der Stifte: wählt einen Stift |

Ein Programm hat bis zu 8 Kostüme, die alle Sprites gemeinsam nutzen. Ein Sprite mit Kostüm ist 16×16 groß. Auf der Bühne wird x auf ein ganzes Byte gerundet (2 Pixel in Modus 0, 4 in Modus 1).

## Alle Blöcke

In der Spalte „Block“ erscheinen die Felder wie im Skript.

### EVENTS

| Block | Was er tut |
|---|---|
| WHEN START *sprite* | das Skript startet zusammen mit dem Programm |
| WHEN KEY *key sprite* | startet, wenn die Taste gedrückt wird |
| WHEN CLICKED *sprite* | startet mit einem Klick auf das Sprite (oder auf die Bühne) |
| WHEN I RECEIVE *n sprite* | startet, wenn Nachricht n gesendet wird |
| SEND *n* | sendet Nachricht n (n kann auch eine Variable sein) |

> Eine Nachricht startet jedes Skript, das auf sie wartet, **von vorne**, auch wenn es schon lief. Deshalb sollte ein Skript keine Nachricht an sich selbst senden.

### CONTROL

| Block | Was er tut |
|---|---|
| FOREVER … END | wiederholt für immer |
| REPEAT *n* … END | n-mal |
| IF *a comparison b* … [ELSE …] END | führt die Zeilen nur aus, wenn der Vergleich zutrifft |
| WAIT *n* | wartet n Frames (50 = eine Sekunde) |
| WAIT UNTIL *a comparison b* | wartet, bis der Vergleich zutrifft |
| STOP | beendet dieses Skript |

## MOTION

| Block | Was er tut |
|---|---|
| GO TO *x y* | das Sprite geht zu x, y |
| CHANGE X *n*, CHANGE Y *n* | bewegt um n |
| SPEED *vx vy* | setzt seine Geschwindigkeit |
| MOVE | ein Schritt mit seiner Geschwindigkeit |
| BOUNCE | kehrt am Rand der Bühne die Geschwindigkeit um |

## LOOKS/SOUND

| Block | Was er tut |
|---|---|
| COLOUR *n* | der Stift des Rechtecks |
| COSTUME *n* | trägt Kostüm n (1–8. 0 ändert nichts) |
| NEXT COSTUME | das nächste, und nach dem letzten wieder das erste |
| SHOW, HIDE | |
| PRINT *value* AT *x y* | gibt eine Zahl aus |
| STYLE *n* | der Stil der folgenden PRINT und TEXT: 1 fett, 2 kursiv, 4 unterstrichen, addiert (7: alle), 0 normal |
| NOTE *n* FOR *frames* | spielt eine Note (MIDI-Nummer: 60 = C, 69 = A 440 Hz) und wartet, bis sie zu Ende ist |
| MUSIC *"Name"* | spielt das Lied *Name*`.MUS` von der Diskette des Programms, hinter ihm, mit dem Programm MUSIC (auf A: oder B:). Es läuft nach dem Ende des Programms weiter, bis zu MUSICs STOP (F6). Es braucht eine Speichererweiterung; ohne sie: „ERST EIN ANDERES PROGRAMM SCHLIESSEN“. Ein Name, den es nicht gibt (oder leer), hält die Musik an |

## VARIABLES

| Block | Was er tut |
|---|---|
| SET *variable* TO *value* | |
| CHANGE *variable* BY *value* | Addition |
| DECREASE *variable* BY *value* | Subtraktion |
| MULTIPLY *variable* BY *value* | |
| DIVIDE *variable* BY *value* | ganzzahlige Division |
| REMAINDER *variable* BY *value* | der Rest der Division |
| SAVE VARS *group* TO *"name"* | schreibt die Variablen in eine Textdatei (siehe unten) |
| LOAD VARS *group* FROM *"name"* | liest sie zurück |

### WINDOWS

| Block | Was er tut |
|---|---|
| WINDOW *"title" width height* | ein Fenster in der Mitte der Bühne, wie die Dialoge von V-OS. Ein neues Fenster nimmt den Platz des vorigen und seiner Schaltflächen ein |
| BUTTON *"text"* AT *x y* | eine System-Schaltfläche. Sie wird mit einem Klick oder mit TAB und RETURN gedrückt. Die Schaltflächen werden ab 0 nummeriert, in der Reihenfolge, in der sie erscheinen |
| WHEN BUTTON *sprite* | startet, wenn irgendeine Schaltfläche gedrückt wird. Der Wert BUTTON (P) sagt, welche |
| TEXT *"text"* AT *x y* | schreibt Text |
| LABEL *button* TO *"text"* | ändert den Text einer Schaltfläche |

Ist ein Fenster offen, zählen TEXT, BUTTON und PRINT ab der linken oberen Ecke seines Inneren. Das Fenster wird an seinem Titel verschoben, zusammen mit seinen Schaltflächen.

## Variablen

Es gibt 128 Variablen, die alle Skripte gemeinsam nutzen, mit den Namen A–H, A1–H1, …, A15–H15. Am Anfang sind alle 0, und sie enthalten ganze Zahlen von −32768 bis 32767.

### Speichern und Wiederherstellen (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` schreibt alle 128 in die Datei `GAME.TXT`. Das `.TXT` wird angehängt, wenn der Name keinen Typ hat.
- Mit einer Gruppe (`A-H`, `A1-H1`, …) schreibt es nur diese 8.
- Die Datei ist Text, eine Zeile pro Variable:

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` ändert die Variablen, die die Datei nennt. Mit einer Gruppe ändert es nur die, die auch in der Gruppe sind. Zeilen, die es nicht versteht, werden ignoriert, und fehlt die Datei, ändert sich nichts.
- So kann eine Datei alle Variablen wiederherstellen oder nur eine Achtergruppe, z. B. die gespeicherten Positionen eines Spiels.

## Dateien

- **`NAME.VBS`:** das Programm, so wie du es bearbeitest. OPEN öffnet es, und bis zu 5K werden gelesen.
- **`NAME.VBN`:** das startbereite Programm. RUN schreibt es, und es startet auch aus der Ansicht Programme. Es ist der Player (`VBRUN.SYS`) zusammen mit dem Programm. Der Player übersetzt die Blöcke beim Start in Z80-Code, deshalb laufen die Programme schnell.

Eine `.VBS` lässt sich auch aus Text auf dem PC erzeugen, mit `tools/mkvbs.py`. Die Syntax ist am Anfang der Datei beschrieben, und Beispiele sind `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS).

## Grenzen

| | |
|---|---|
| Zeilen (Blöcke) | 240 |
| Sprites | 6 |

| | |
|---|---|
| Skripte (`WHEN …`) | 16 |
| Variablen | 128 |
| REPEAT in einem Programm | 64 (darüber hinaus teilen sie sich einen Zähler) |
| Ineinander verschachtelte offene Blöcke | 16 |
| Texte | 32, je bis zu 14 Zeichen |
| Kostüme | 8, 16×16 |
| Schaltflächen | 64 |
| Code | mindestens 4.000 Bytes, auch für das größte Programm. Was nicht hineinpasst, wird ignoriert |

## Tipps

- **Skripte der Bühne:** Die Bewegungs- und Aussehen-Blöcke tun nichts, und X, Y, TOUCHING und AT EDGE ergeben 0.
- **TOUCHING:** trifft nur zu, wenn beide Sprites sichtbar sind.
- **Zeitablauf:** Die Skripte laufen „zusammen“. Jedes Skript gibt seinen Zug am Ende jeder Schleife und bei jedem WAIT ab. Soll ein Skript auf ein anderes warten, nimm eine Variable und `WAIT UNTIL` (siehe MINES).
- **Fertige Beispiele:** CALC, MINES, FLAP und TICKER lassen sich in Studio öffnen (OPEN).

# 6. Blöcke auf dem PC: der Web-Editor und `vbc.py`

Neben Studio, das auf dem CPC läuft, gibt es auch einen Block-Editor für den PC. Die Sprache ist dieselbe, mit ein paar zusätzlichen Blöcken: Listen, Rauschen, Sprite-Größe, farbiger Text.

## Der Editor

Öffne `blocks/editor.html` in einem Browser. Er basiert auf Blockly und gibt es auf Griechisch oder Englisch.

- **Kategorien:** Ereignisse, Steuerung, Bewegung, Aussehen, Text, Operatoren, Fühlen, Variablen, Listen und Fenster.
- **Sprites:** Rechtecke oder Kostüme. Sie werden in einer Tabelle festgelegt, und die Bühne zeigt sie in den Farben des CPC. Sie lassen sich mit der Maus verschieben.
- **Dateien:** Er öffnet und speichert `.vbp`-Dateien. Sie sind JSON mit dem ganzen Programm, deshalb kann man sie auch von Hand schreiben.

## Übersetzen

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` schreibt `build/NAME.ASM`, mit einem Kommentar für jeden Block. Es assembliert sie mit RASM zu `NAME.VBN`, mit einer Relokationstabelle, damit sie neben anderen Programmen läuft, und mit `--dsk` legt es sie auf die Diskette.

| Option | Was sie tut |
|---|---|
| `-o build` | Ausgabeordner |
| `--asm-only` | nur die `.ASM` |
| `--dsk build/vos.dsk` | auch auf die Diskette |

## Das Format `.vbp`

Die vollständige Beschreibung steht am Anfang von `blocks/vbc.py`: Ereignisse, Befehle, Werte, Kostüme, Symbol. Kurz gesagt:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variablen und Listen:** Sie haben Namen, keine Buchstaben. Listen haben eine feste Höchstlänge.
- **Symbol:** `"icon"` gibt ein Symbol für die Ansicht Programme an.
- **Ein Programm mit vielen ähnlichen Teilen:** Ein Python-Skript kann es schreiben. Siehe `tools/mkbricks.py`, das `bricks.vbp` mit 32 Steinen schreibt.

## Beispiele

In `blocks/examples/`:

| Datei | Programm |
|---|---|
| `pong.vbp` | PONG |
| `catch.vbp` | CATCH |
| `starcat.vbp` | STARCAT: Kostüme, eine Melodie aus einer Liste |
| `counter.vbp` | COUNTER: ein Fenster und Schaltflächen |
| `bricks.vbp` | BRICKS (geschrieben von `tools/mkbricks.py`) |

Die Dateien `*.vbt` sind Studio-Programme als Text, für `tools/mkvbs.py`. Siehe Kapitel 5.

## Studio oder Web-Editor?

| | Studio | Web-Editor |
|---|---|---|
| Wo | auf dem CPC | auf dem PC |
| Datei | `.VBS` | `.vbp` |
| Variablen | 128, A–H15 | mit Namen |
| Listen, Rauschen, Sprite-Größe | nein | ja |
| SAVE/LOAD VARS | ja | nein |
| Textstil, `.MUS`-Musik | ja | ja |
| Das Programm | braucht den ganzen Speicher | klein, läuft neben anderen |

# 7. Das V-OS-SDK: Programme in Assembler

Mit diesem SDK schreibst du deine eigenen Programme in Z80-Assembler. Sie laufen in V-OS wie DEMO oder CLOCK: Sie erscheinen in den Ansichten Programme, Schreibtisch und Commander, und sie haben ihr eigenes Menü, Schaltflächen, Dialoge, Ton und Dateien. Sie laufen auch neben anderen Programmen.

## Schnellstart

```
make new-app NAME=hello
```

Das legt den Ordner `myapps/hello/` aus `sdk/template/` an. Er enthält drei Dateien:

- `hello.asm`: das Programm. Die Pfeile bewegen ein Kästchen, es gibt eine Schaltfläche BEEP, einen Sekundenzähler und ein eigenes Menü (ABOUT, BACK) neben ANSICHT und HILFE.
- `texts.txt`: seine Texte, auf Englisch und Griechisch.
- `icon.txt`: sein Symbol, 16×16, für die Ansicht Programme.

```
make app NAME=hello
```

Das baut `build/HELLO.BIN` und legt es auf die Diskette der Programme, `build/apps.dsk`. `make run` legt sie in B: ein; im Launcher zeigt TAB das Laufwerk B:.

```
make run
```

Das öffnet die Diskette in Caprice32. HELLO steht in der Ansicht Programme, auf der letzten Seite.

- Ein einfaches `make` baut alle Programme in `myapps/` und legt sie zusammen mit V-OS auf die Diskette.
- Der Name hat 1–8 Kleinbuchstaben oder Ziffern und beginnt mit einem Buchstaben. Er darf nicht der Name eines V-OS-Programms sein (z. B. demo).
- Du brauchst RASM, Python 3 und iDSK, wie für ganz V-OS.
- In Caprice32 startest du das Programm selbst, aus der Ansicht Programme.

Ein größeres Programm:

- Leg weitere `.asm`-Dateien in seinen Ordner (z. B. `art.asm` mit Sprites) und hol sie mit `include "art.asm"` herein. Der Build findet sie und baut das Programm neu, wenn sie sich ändern.
- Was du in den Unterordner `disc/` legst (z. B. `disc/LEVEL1.DAT`), kommt unverändert auf die Diskette, für `VOS_LOAD`.
- Mit einem Unterordner `owndisc/` bekommt das Programm auch eine eigene Diskette, `build/<name>.dsk`, mit der `.BIN` und den Dateien aus `owndisc/`. Es kommt nicht auf `apps.dsk`. Du legst sie in B: ein und startest es aus dem Launcher (TAB, das andere Laufwerk). Von welchem Laufwerk es gestartet wurde, damit es seine Dateien von dort lädt, steht in `VOS_SELF`: `db Laufwerk`, dann sein Name (11 Bytes).
- **Eine Datei zum Öffnen (Version 14):** wenn RETURN auf dem Schreibtisch oder im Commander eine Datei in deinem Programm öffnet, enthält `VOS_OPEN` (`#B000`) `db 1, Laufwerk, 11-Byte-Name` (0: keine). Lies es in `init`, mit A = 0 und auch mit A = 2 (das Programm lief schon), und schreibe danach 0 hinein. Die Typen und ihre Programme stehen

in der Tabelle `ft_open` des Kernels (`src/kernel/files_ext.asm`).

- Ein Programm mit Relocation-Tabelle (wie die des SDK) darf bis zu 12K groß sein. Für den Rest lädst du Dateien in die Daten-Bank.

## Wie ein Programm funktioniert

Ein Programm ist eine `.BIN`, gebaut für `VOS_APP_ADDR` (`#4000`). Es beginnt so:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
        jp init                 ; +0
        jp event                ; +3  HL -> das Ereignis
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 das Symbol (optional)
```

- **`init`** setzt Modus, Palette, Menü und Schaltflächen, schaltet bei Bedarf `VE_FRAME` ein und zeichnet alles.
  - A = 0: ein frischer Start.
  - A = 2: Das Programm kehrt aus einer Ansicht oder einem anderen Programm zurück. Sein Zustand ist so, wie es ihn verlassen hat; es setzt das oben Genannte erneut und zeichnet.
- **`event`** wird für jedes Ereignis aufgerufen und kehrt sofort zurück. Ein Programm wartet nie selbst (keine Warteschleife). Für Bewegung oder Zeit `VOS_TICKS` mit A = 1, dann kommt 50-mal pro Sekunde ein `VE_FRAME`.
- **`redraw`** zeichnet alles neu, z. B. wenn sich die Sprache ändert oder das Zubehörprogramm sich schließt.
- **Beenden:** `VOS_EXIT`, das zu der Ansicht zurückkehrt, die es gestartet hat.

### Die Reihenfolge in `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; die Frames
        push hl
        call VOS_UI_EVENT               ; Menüs, Schaltflächen, CONTROL+Buchstabe
        pop hl
        or a : jr nz,command            ; A = der Befehl
        call VOS_MENU_OPEN : ret nz     ; Menü offen: die Tasten gehören ihm
        ...                             ; deine eigenen Tasten und Klicks
command: ...
        jp VOS_COMMAND                  ; was nicht deins ist (VIEW, HELP)
```

**Solange ein Menü offen ist, zeichne nicht.** Sonst zeichnest du darüber. Bei `VE_FRAME` aktualisiere den Zustand, aber zeichne nur, wenn `VOS_MENU_OPEN` Z ergibt. Dialoge (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) haben ihre eigene Schleife, deshalb kommt kein Ereignis an, solange sie offen sind.

### Die Ereignisse

`HL` → 8 Bytes: `db type, code, char, mods; dw x, y`.

| type | was | code / char / x, y |
|---|---|---|

| | | |
|---|---|---|
| VE_KEY_DOWN 1, VE_KEY_REPEAT 3, VE_KEY_UP 2 | Taste | code = Tastennummer (VK_*, Tastaturreihe·8 + Bit), char = das Zeichen in der Sprache des Pakets |
| VE_PTR_DOWN 5, VE_PTR_UP 6, VE_PTR_MOVE 4 | Zeiger | x, y in Pixeln des Modus |
| VE_FRAME 8 | Frame (mit VOS_TICKS) | — |
| VE_LANG 7 | die Tastatursprache hat sich geändert | — |

### Drei Arten von Programmen

- **Mit Relokationstabelle (die Voreinstellung des SDK):** `make app` baut es zweimal, bei `#4000` und bei `#4100`. `tools/mkreloc.py` fügt die Tabelle aus dem Unterschied hinzu.
  - Es läuft neben anderen Programmen, in den 12K ab `#5000`, und macht dort weiter, wo es aufgehört hat (F6 zeigt die laufenden Programme).
  - Seine Daten müssen in seinen eigenen Bytes liegen (Labels, `ds`), nie an festen Adressen.
  - Gibt `mkreloc.py` einen Fehler aus, wird irgendwo eine Adresse so benutzt, dass sie die Relokation nicht übersteht. Z. B. `ld a,label & 255`, oder nur das niedrige Byte einer Adresse.
- **Ohne Tabelle:** Es nimmt die ganze Bank (bis zu 16K, Code und Daten), läuft aber nur allein, außer es gibt eine Speichererweiterung. Dafür brauchst du eine eigene Regel im Makefile (siehe `STUDIO.BIN`).
- **Zubehörprogramm:** bis zu 4K, mit Tabelle. Es öffnet sich mit F5 über allem, und `init` bekommt A = 1. Siehe `apps/clock/clock.asm` und die Beschreibung in `src/api/vos_api.inc`.

### Regeln

- **Register:** Aufrufe verändern alle Register außer IX. Argumente kommen in A, BC, DE, HL (IX für `VOS_RECT` und `VOS_BLIT`). Ergebnisse kommen in A oder HL zurück. Bei einem Fehler ist Carry gesetzt und A = Fehlernummer, die `VOS_ERROR` anzeigt.
- **Speicher:** Blende keine andere Bank bei `#4000` ein, denn dann verschwindet das Programm unter dir. Für 16K mehr gibt es die Daten-Bank (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). Sie wird mit Ansichten und anderen Programmen geteilt und behält ihre Daten nur, solange das Programm läuft. Mit einer Speichererweiterung (API 21) bekommt jedes Programm eine eigene, die ihren Inhalt bis zu seinem Ende behält, was auch immer dazwischen läuft; `VX_DATA` sagt, welche gilt (A = 1: eine eigene).
- **Koordinaten:** x in Pixeln des Modus (Modus 0: 0–191, Modus 1: 0–383), y von 0 bis 258. Die Menüleiste belegt y 0–11.
- **RASM:** Bei Labels spielt Groß- und Kleinschreibung keine Rolle (`BOX` und `box` sind dasselbe). Hex wird mit `#` geschrieben und nicht mit `&`.

## Die Datenformate

- **Menüleiste** (`VOS_SET_BAR`): `db count; dw menu, ...`. Die gemeinsamen Menüs VIEW und HELP (auf dem Bildschirm ANSICHT und HILFE) kommen von `VOS_MENU_VIEW` und `VOS_MENU_HELP`.
- **Menü:** `db title, count` und für jeden Eintrag `db text, command, letter, 0`.

    - text: `VOS_SEPARATOR` für eine Linie
    - command: 1.., was `VOS_UI_EVENT` zurückgibt
    - letter: Kurzbefehl CONTROL+Buchstabe, oder 0

- **Schaltflächen** (`VOS_SET_WIDGETS`, dann `VOS_DRAW_WIDGETS`): `db count`, und für jede Schaltfläche `dw x, y; db Breite in Zeichen, text, command, 0`.
- **Dialog** (`VOS_DIALOG` → A = Schaltfläche, 0 die erste, ESC ergibt die letzte): `db title, text, Anzahl der Schaltflächen (1-3), button, ...`. Für die Schaltflächen `VM_OK`, `VM_YES` und `VM_NO`.
- **Texte** (`VOS_TEXTS`): erzeugt von `tools/mktexts.py` aus `texts.txt`, mit Zeilen `NAME | English | Greek`. Beide Spalten werden in UTF-8 geschrieben und werden zu Bytes des V-OS-Zeichensatzes (`tools/voscs.py`), der Griechisch und die lateinischen Buchstaben von Französisch, Spanisch, Deutsch und Italienisch enthält.
    - Es liefert `T_NAME` (`VT_FIRST` = `#C0` und aufwärts) und die Tabelle `texts:`. Bis zu 64 Texte.
    - Ein `|` ohne Leerzeichen davor und danach beginnt in Dialogen eine neue Zeile.
    - Texte gehen überall dorthin, wo ein „Text“ verlangt wird (Menüs, Schaltflächen, Dialoge). Mit `VOS_MSG` bekommst du ihre Adresse für `VOS_PRINT`.
- **Palette** (`VOS_PALETTE`): 17 Firmware-Farben (0–26): 16 Stifte und der Rand. In Modus 1 zählen die Stifte 0–3.
- **Rechteck** (`VOS_RECT`, IX →): `dw x, y, w, h; db pen`.
- **Symbol:** 16 Zeilen mit 16 Zeichen in `icon.txt`. Jedes Zeichen ist ein Stift in Hex (Modus 0, Palette der Ansicht Programme) oder `.` für transparent. Die Ansicht Programme liest es von der Diskette: solange das Programm läuft, sind die 256 Bytes nach "IC" frei für Puffer (PAINT legt dort seine Zeilen ab).
- **Dateinamen** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 Bytes, `"NAME EXT"`. Laufwerk 0 = A:, 1 = B:.

## Die API (Version 22)

| Aufruf | Eingabe → Ausgabe | Was er tut |
|---|---|---|
| `VOS_EXIT` | — | Ende, zurück zu der Ansicht, die es gestartet hat |
| `VOS_TICKS` | A = 1 / 0 (+ `VOS_PTR_KEYS`) | `VE_FRAME` 50-mal pro Sekunde, oder keines. Mit `VOS_PTR_KEYS` (2) bewegen die Pfeile und COPY auch den Zeiger und klicken |
| `VOS_FRAMES` | → HL (und A Bits 16–23) | Frames seit dem Start |
| `VOS_RANDOM` | → A | 0–255 |
| `VOS_COMMAND` | A = Befehl | die gemeinsamen Befehle von VIEW und HELP (6–19). Nummeriere deine eigenen Befehle ab `VC_APP` (32), sonst startet ein Eintrag von VIEW oder HELP einen deiner Befehle |
| `VOS_ERROR` | A = Fehler | Fehlerdialog |
| `VOS_MODE` | A = 0 / 1 | Modus 0 (192×259, 16 Stifte) oder 1 (384×259, 4 Stifte) |
| `VOS_PALETTE` | HL → 17 Farben | |
| `VOS_CLS` | A = Stift | löscht den Bildschirm |

| | | |
|---|---|---|
| `VOS_PLOT` | HL = x, DE = y, A = Stift | ein Pixel |
| `VOS_RECT` | IX → Rechteck | gefülltes Rechteck |
| `VOS_COLOURS` | A = Textstift (+ `VOS_BOLD`, `VOS_ITALIC`, `VOS_UNDERLINE`), C = Hintergrund | für `VOS_PRINT` und `VOS_CHAR`. Die Stile entstehen aus derselben Schrift, für jede Sprache |
| `VOS_PRINT` | HL = x, DE = y, BC → Text, 0 | |
| `VOS_CHAR` | HL = x, DE = y, A = Zeichen | |
| `VOS_MSG` | A = Text → HL | die Adresse eines Textes (`T_*` oder eines Systemtextes) |
| `VOS_SET_BAR`, `VOS_DRAW_BAR` | HL → Menüleiste | |
| `VOS_UI_EVENT` | (das aktuelle Ereignis) → A | Befehl von Menü, Schaltfläche oder Kurzbefehl, 0 für keinen |
| `VOS_DIALOG` | HL → Dialog → A | |
| `VOS_INPUT` | A = Titel, HL → Text, C = Maximum → Carry bei Abbruch | Texteingabe (braucht C + 2 Bytes) |
| `VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS` | HL → Schaltflächen | |
| `VOS_MENU_VIEW`, `VOS_MENU_HELP` | → HL | die gemeinsamen Menüs |
| `VOS_CATALOG` | A = Laufwerk → A = Dateien, HL → Liste | je 16 Bytes: 11 der Name, bei +12 die Größe in K. Mit A = Laufwerk + `VOS_CAT_EXT` und HL → `"EXT"` nur die Dateien mit dieser Endung |
| `VOS_FILE_PICK` | A = Titel → A = Datei, Carry bei Abbruch | Auswahldialog, nach `VOS_CATALOG` |
| `VOS_LOAD` | A = Laufwerk, HL → Name, DE = wohin → BC = Länge | |
| `VOS_SAVE` | A = Laufwerk, HL → Name, DE = ab, BC = Länge | Binärdatei, ersetzt die alte. Mit A + `VOS_LOAD_IX` (#40, Version 20) sagt ihr Header, dass sie bei IX lädt, nicht bei DE (eine `.SCR`: IX = #C000) |
| `VOS_DELETE` | A = Laufwerk, HL → Name | |
| `VOS_TEXTS` | HL → Texttabelle | die Texte des Programms |
| `VOS_MENU_OPEN` | → NZ, wenn ein Menü offen ist | |
| `VOS_KEY_HELD` | A = Taste → NZ, solange gedrückt | für Spiele |
| `VOS_BLIT` | IX → `dw x, y; db bytes, lines; dw source; db source stride` | Bild: Paare (Maske, Pixel) |
| `VOS_SOUND` | A = Kanal 0–2, HL = Periode (Bit 15: Rauschen), C = Lautstärke 0–15, B = Frames | eine Note; 440 Hz ist Periode 142 |
| `VOS_SPLIT` | HL → `dw y1, y2; db 8 colors` (0 = schaltet ab) | gemischter Bildschirm: Modus 0 zwischen y1 und y2 |
| `VOS_RUN` | A = Laufwerk, HL → Name | startet ein anderes Programm, und danach startet dieses wieder |

| | | |
|---|---|---|
| VOS_KEEP | → HL → 16 Bytes | Speicher, der zwischen Programmen erhalten bleibt |
| VOS_BANK_GET, VOS_BANK_PUT | HL = ab, DE = nach, BC = Bytes | Kopieren aus der und in die Daten-Bank |
| VOS_BACKGROUND | A = 1 / 0 | läuft weiter oder friert ein, wenn es hinter einem anderen liegt |
| VOS_DRAG | HL = x, DE = y, BC = w, A = h → Carry: neue HL, DE | ein Fenster an seinem Titel verschieben |
| VOS_FIT | dasselbe → HL, DE | holt ein Fenster in den Bildschirm hinein |
| VOS_EXT | A = ein VX_*-Aufruf, BC, DE, HL wie er es will → A, HL, C, DE, Carry | die Aufrufe nach der Tabelle (Version 15), unten |

**Die Aufrufe von VOS_EXT:** VX_VERSION → A = API, HL = V-OS-Version (BCD). VX_FREE C = Laufwerk → A = freie K. Unbekannt: Carry VX_CLIP_PUT HL = Text, BC = Bytes → die Zwischenablage (bis 1K). VX_CLIP_GET DE = wohin, BC = Platz → HL = Größe (Version 16). VX_M4 HL → ein Befehl für die M4-Karte (db Größe, dw Befehl, Daten), DE → die Antwort, BC = Bytes; Carry: keine M4-Karte (Version 17). VX_MEM → HL = der RAM in K, DE = die freien K für Programme (Version 18). VX_BEHIND DE → db Laufwerk, 11-Byte-Name einer Datei: ihr Programm (nach Typ) startet hinter diesem, mit init A = 3 und der Datei in VOS_OPEN (Version 19; MUSIC kennt es). VX_DATA → A = 1, wenn die Daten-Bank die eigene ist (Speichererweiterung), 0, wenn es die geteilte Bank 4 ist (Version 21; NOTEPAD hält so Text bis 16K). VX_DB C = ein DB_*-Aufruf, DE, HL → eine Datenbank (Version 22, unten). VX_UTF8 HL → ein Text, BC = Bytes, E = 0 im Speicher des Programms oder VOS_DATA in seiner Datenbank-Bank: ist er UTF-8 (vom PC oder aus dem Web), wird er dort zum V-OS-Zeichensatz → BC = Bytes jetzt, A = 1; A = 0: er war kein UTF-8 und bleibt, wie er war (Version 23; so machen es NOTEPAD und READER). VX_CALC C = ein CALC_*-Aufruf, DE → ein Block, HL → Text → Formeln und Zahlen (Version 24, unten).

## Datenbanken (Version 22)

VOS_EXT mit A = VX_DB und C = einem DB_*-Aufruf arbeitet mit einer Datenbank: einer Datei NAME.VDB aus 1K-Seiten, so groß wie die Diskette erlaubt (bis 255K). Sie hat bis zu 8 Tabellen; eine Tabelle hat bis zu 10 Felder, Text (bis 255 Zeichen) oder eine Zahl (2 Bytes), und kann ein Schlüsselfeld haben (Text bis 64 Zeichen, oder eine Zahl) mit einem Index, damit FIND schnell ist. Ein Datensatz sind seine Felder hintereinander; seine Kennung (rid) ist eine 16-Bit-Zahl. Eine Datenbank ist jeweils offen, für das Programm, das sie geöffnet hat. Jeder Aufruf schreibt seine Änderungen auf die Diskette, bevor er zurückkehrt. Der Kernel lädt die Maschine, DB.SYS, beim ersten Mal von Laufwerk A:.

| Aufruf | Ein → aus |
|---|---|
| DB_CREATE, DB_OPEN | DE → db Laufwerk, 11-Byte-Name ("STREETS VDB"); OPEN → A = 1: Latin-1 (von V-OS 0.15) |
| DB_CLOSE | |
| DB_TABLE | DE → Name (8), Felder, Schlüsselfeld (0: keins), dann jedes Feld: Name (8), Typ, Länge → A = Tabelle |
| DB_USE | E = die Tabelle, die die Aufrufe unten nutzen |
| DB_INFO | E = Tabelle, HL → 124 Bytes: ihre Beschreibung → A = Tabellen |
| DB_ADD | HL → Datensatz → DE = rid |
| DB_GET, DB_PUT | DE = rid, HL → der Datensatz |
| DB_DEL | DE = rid |
| DB_FIND | HL → Schlüssel wie im Datensatz (Platz für einen Datensatz), E = zählende Bytes (0: alle) → DE = der erste Datensatz mit diesem Schlüssel oder danach |

| | |
|---|---|
| DB_FIRST, DB_LAST, DB_NEXT, DB_PREV | HL → Platz für den Datensatz, oder 0 → DE = rid, nach Schlüssel; Carry am Ende |
| DB_COUNT | → HL = Datensätze |

Fehler: Carry und A (DB_ERR_*, und die Fehler der Diskette), die VOS_ERROR zeigt. Text wird gespeichert, wie er getippt wurde, im V-OS-Zeichensatz, in allen Sprachen demselben: eine Datenbank kann Griechisch, Französisch und Deutsch zusammen enthalten. Nur eine Datenbank von V-OS 0.15 mit lateinischen Buchstaben (Latin-1) ist anders: DB_OPEN gibt dann A = 1; zeige DB_ERR_ALPHABET mit VOS_ERROR. Sortieren und FIND vergleichen Bytes: in einem DB_TEXT-Schlüssel sind Groß- und Kleinbuchstaben, und solche mit Akzent, verschieden. Ein DB_ALPHA-Feld (Typ 2, ab Version 23) ist Text wie DB_TEXT, wird als Schlüssel aber ohne Groß-/Kleinschreibung und Akzente sortiert und gefunden: ΕΛΕΝΗ, Ελένη und ελένη, oder Élodie und ELODIE, sind dasselbe; der Datensatz behält den Text, wie er getippt wurde. Beispiele sind STREETS (myapps/streets), ein Straßenverzeichnis, und DATABASE (myapps/database), ein ganzes Datenbankprogramm mit Tabellen, Liste und Formular; tools/vdb.py zeigt und prüft eine .VDB-Datei auf dem PC.

### Formeln (Version 24)

VOS_EXT mit A = VX_CALC und C = einem CALC_*-Aufruf berechnet Formeln, wie eine Tabellenkalkulation. Der Kernel lädt die Engine, CALC.SYS, beim ersten Mal von Laufwerk A:. Eine Zahl hat 32 Bit, in Hundertsteln (1234 ist 12.34), von -21474836.46 bis 21474836.46. Eine Formel hat + - * /, Klammern, Zahlen (12, 3.5, .25; eine dritte Dezimalstelle rundet), Zellen (A1 bis Z255) und die Funktionen SUM, AVG, MIN, MAX, COUNT (von Werten, Zellen und Bereichen wie B2:D9, getrennt durch , oder ;), ABS(x), INT(x) (abwärts auf eine ganze Zahl) und ROUND(x) oder ROUND(x, 0–2), groß oder klein geschrieben. Produkte und Quotienten werden auf das Hundertstel gerundet.

DE → der Block des Programms, CB_SIZE (10) Bytes: CB_RESULT (eine Zahl, 4 Bytes), CB_COLS, CB_ROWS, CB_GRID (dw), CB_BANK, CB_AT. Die Zellen sind ein Raster des Programms: CB_COLS × CB_ROWS Zahlen zu 4 Bytes, Zeile für Zeile, bei CB_GRID, in seiner Daten-Bank, wenn CB_BANK = VOS_DATA. Eine Zelle CALC_NONE hat keine Zahl (leer oder Text: 0 in + - * /, von den Funktionen ausgelassen); CALC_BAD hat einen Fehler.

| Aufruf | Eingabe → Ausgabe |
|---|---|
| CALC_EVAL | HL → Formel, 0 (mit = davor oder nicht) → CB_RESULT und DE:HL (DE der hohe Teil) |
| CALC_NUMBER | HL → Text: nur eine Zahl? → wie CALC_EVAL |
| CALC_TEXT | CB_RESULT → Text bei HL (Platz für 13 Bytes: -12.5), A = seine Länge |
| CALC_CELL | HL → Text: eine Zelle (b12) am Anfang? → D = Spalte, E = Zeile (ab 0), A = ihre Zeichen |

Fehler: Carry und A (CALC_ERR_*: 30 falsche Formel, 31 Division durch null, 32 Zahl zu groß, 33 Zelle außerhalb des Rasters, 34 eine benutzte Zelle hat einen Fehler, 29 CALC.SYS ist nicht auf A:), die VOS_ERROR zeigt; dann ist CB_RESULT = CALC_BAD und CB_AT = die Stelle im Text. Das Beispiel ist SHEET (myapps/sheet), eine Tabellenkalkulation, die ihre Werte in ihrer Daten-Bank hält und die Formeln nach jeder Änderung neu berechnet.

Alle Details, z. B. zum Hintergrund, zum Zubehörprogramm und zu VOS_SPLIT, stehen in src/api/vos_api.inc. Beispiele findest du in apps/demo/demo.asm (Modus 0, Bewegung), apps/clock/clock.asm (Zubehör) und apps/mixed/mixed.asm (gemischter Bildschirm). Ein ganzes Spiel mit Sprites, einem Level aus einer Datei und Ton ist myapps/vrouves/.

Ein KI-Assistent, der ein Programm schreibt, sollte AI.md lesen (auf Englisch): dieselben Regeln, kürzer, mit den Fallen von RASM und der Datenbank.

## Testen

make test-sdk baut die Vorlage als TEMPLATE.BIN und öffnet sie in Caprice32 aus der Ansicht

Programme, mit Tastendrücken, die `src/kernel/test_script.asm` festlegt (Skript 15). Es macht Screenshots in `build/shots`: seinen Bildschirm, das Kästchen nach der Bewegung, sein Menü, den Dialog ABOUT und die Rückkehr zur Ansicht Programme.

Für deinen eigenen automatischen Test kopiere Skript 15 und das Ziel `test-sdk` des Makefiles.

# 8. V-OS entwickeln

Für alle, die an V-OS selbst arbeiten. Programme *für* V-OS beschreibt das SDK.

## Was du brauchst

- RASM, den Assembler
- iDSK, für die Disketten-Images
- Python 3, für die Werkzeuge. `make docs` braucht außerdem `markdown` und `pymupdf`.
- GNU make 4.3 oder neuer
- zum Testen Caprice32 (snap `caprice32`)

## Build

| Befehl | Was er tut |
|---|---|
| `make` | `build/vos.dsk` (A:): der Kernel, die Ansichten, die Sprachen, DEMO, MIXED, STUDIO, CLOCK und die `.VBS`-Quellen; `build/apps.dsk` (B:): die anderen Programme und `myapps/*`; und die Diskette jedes Programms mit `owndisc/`. Passt eine Datei nicht auf eine echte Diskette (180K), bricht es mit einem Fehler ab |
| `make run` | öffnet die Diskette in Caprice32 (`RUN"VOS"`) |
| `make shot` | startet ohne Fenster und macht einen Screenshot |
| `make docs` | das Handbuch als PDFs: `docs/V-OS.pdf` (Griechisch) und `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it` |
| `make windows` | `build/V-OS-windows.zip`: `V-OS.exe` für Windows, mit CPCEMU 3.0.2 (CPC 6128, 576K, AMX-Maus, M4-Karte), `vos.dsk` in A:, `apps.dsk` in B: und die eigenen Disketten der Programme daneben. V-OS startet von selbst. Braucht WSL und das portable Zip von CPCEMU (`CPCEMU_ZIP=…`) |
| `make linux` | `build/V-OS-linux.tar.gz`: das Skript `V-OS` für Linux (x86_64), mit CPCEMU 2.5 (3.0.2 gibt es für Linux nicht) und denselben Einstellungen und Disketten. Braucht SDL2 auf dem System und das Linux-tar.gz von CPCEMU (`CPCEMU_TGZ=…`) |
| `make songs` | die Songs von Arkos Tracker 3 als `.MUS`-Dateien in `myapps/music/owndisc` (`tools/mkmusic.py`; braucht Arkos Tracker 3, `ARKOS=…`) |
| `make new-app NAME=x`, `make app NAME=x` | dein eigenes Programm (SDK) |
| `make clean` | löscht `build/` |

## Aufbau des Repos

| Ordner | Inhalt |
|---|---|
| `src/vos.asm` | die Wurzel des Builds: Lader und Kernel, Schreiben auf die Diskette |
| `src/boot/` | der Lader, den AMSDOS bei `#4000` startet, Erkennung der RAM-Erweiterung |
| `src/kernel/` | Vektoren, Interrupts, Dateisystem, Ansichten, API (`api.asm`), Einstellungen, Kernel-Erweiterung (`ext.asm`), Testskripte (`test_script.asm`) |
| `src/drivers/` | Gate Array, CRTC, Tastatur, AMX-Maus, FDC, AY |
| `src/gfx/` | Overscan-Grafik, Text, gemischter Bildschirm |
| `src/ui/` | Menüs, Schaltflächen, Dialoge, Texteingabe, Dateiauswahl |

| | |
|---|---|
| `src/shell/` | die Ansichten als Overlays: Schreibtisch, Programme, Commander, Optionen |
| `src/api/ vos_api.inc` | die API für Programme |
| `lang/` | die Sprachen (Texte, Tastaturen, Schriften) |
| `apps/` | die Programme in Assembler: DEMO, MIXED, STUDIO, CLOCK, und ihre Symbole |
| `blocks/` | der Web-Editor, `vbc.py`, `runtime.asm`, der Player von Studio (`player.asm`), Beispiele |
| `sdk/` | das SDK: Anleitung und Programmvorlage |
| `tools/` | Werkzeuge: Schriften, Sprachen, Texte, Symbole, Relokation, `.VBS` aus Text, Screenshots, PDF |
| `docs/` | dieses Handbuch auf Griechisch (`manual/`) und auf Englisch, Französisch, Spanisch, Deutsch und Italienisch (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), die PDFs, die Anforderungen (`REQUIREMENTS.md`) |

## Speicher, kurz gefasst

| Bereich | Was er enthält |
|---|---|
| `#0000-#3DFF` | der Kernel. In Seite null, nach den Restart-Vektoren, die API-Tabelle bei `#0108` |
| bis `#4000` | der Stack des Kernels |
| `#4000-#7FFF` | das „Fenster“ für die Bänke |
| `#8000-#FFFF` | der 32K-Overscan-Bildschirm. In seinen Lücken Puffer des Kernels |

Die Bänke, die in das Fenster kommen:

| Bank | Was sie enthält |
|---|---|
| 4 | Datei- und Ladepuffer, und die Daten-Bank des laufenden Programms |
| 5 | die Ansicht bei `#4000-#4FFF`, und ab `#5000` die Programme mit Relokationstabelle (oder ein Programm ohne Tabelle in der ganzen Bank) |
| 6 | was die Menüs und Dialoge verdecken, und ab `#7000` das Zubehörprogramm |
| 7 | das Sprachpaket, die Listen des Commanders, die englischen Texte, und ab `#7000` die Kernel-Erweiterung |

Mit einer Speichererweiterung nehmen ihre Bänke Programme auf. Die Details, Adresse für Adresse, stehen in `docs/REQUIREMENTS.md` (§4.2 und in jedem Abschnitt „Υλοποίηση M…“ (Umsetzung)).

## Konventionen

- **RASM:** Hex wird `#C000` geschrieben und nicht `&C000`, weil RASM `&` als AND liest. Bei Labels spielt Groß- und Kleinschreibung keine Rolle.
- **Kommentare:** auf Englisch, kurz, wie im vorhandenen Code. Jede Datei beginnt mit einem Block, der sagt, was sie tut.
- **Dokumentation:** auf Griechisch. `MILESTONES.md` (Englisch), `MILESTONES.el.md` (Griechisch) und `docs/REQUIREMENTS.md` haben einen Abschnitt für jeden Meilenstein. Das Handbuch wird auf Griechisch geschrieben und in die anderen fünf Sprachen von V-OS übersetzt.
- **Texte:** Die Systemtexte stehen in `lang/*/messages.txt`, die Texte der Programme in `texts.txt` (für `tools/mktexts.py`).

## Tests

Die Tests starten V-OS in Caprice32 ohne Fenster und machen Screenshots in `build/shots/`

`<delay>/`.

**Wie sie funktionieren:**

- Caprice32 ohne Fenster kann keine Pfeile, kein COPY und keine f-Tasten drücken. Deshalb werden die Tastendrücke von einem besonderen Build erzeugt: Ein `-DTEST_SCRIPT=n` fügt Skript n in `src/kernel/test_script.asm` ein, eine Tabelle von Tasten pro Frame.
- Jedes Ziel `make test-…` baut eine Diskette mit seinem Skript und macht bei jeder Verzögerung in `…_SHOTS` einen Screenshot.
- Die Verzögerung wird in „Z“-Drücken gemessen: jeweils etwa 2,5–3,5 Frames. Die Zuordnung ist nicht fest, deshalb fängst du einen Moment mit zwei oder drei Verzögerungen um ihn herum ein.
- Sobald V-OS auf die Diskette schreibt, schließt sich Caprice32 nicht mehr von selbst, und der Test endet mit einem Timeout. Das ist bei test-vars, test-costume und anderen zu erwarten.
- Die Screenshots siehst du dir selbst an, z. B. zu einem Bild zusammengesetzt mit `ffmpeg … hstack/vstack`.

| Ziel | Was es testet |
|---|---|
| `test-input` | Tasten, Studio: Blöcke, Dateien, Starten |
| `test-calc` | CALC mit Klicks, dann `CALC.VBS` in Studio |
| `test-costume` | Kostüm in Studio, Starten, erneutes Öffnen |
| `test-acc` | die Uhr als Zubehörprogramm |
| `test-multi`, `test-switch`, `test-expand`, `test-behind` | mehrere Programme, F6, Speichererweiterung, Hintergrund |
| `test-drag` | Fenster verschieben |
| `test-langs`, `test-lang` | die Sprachen, Speichern der Sprache |
| `test-mines`, `test-bricks` | die Spiele |
| `test-vars` | die 128 Variablen, SAVE/LOAD VARS |
| `test-sdk` | die SDK-Vorlage |
| `test-apps` | die zwei Disketten, wie sie ausgegeben werden, `vos.dsk` (mit den Test-Tasten) in A: und `apps.dsk` in B:: der Launcher zeigt B:, und PONG läuft von dort |
| `test-vrouves` | VROUVES von seiner Diskette in B:, über den Launcher (TAB): Bauen, Welle 1, das Geld, HIGH SCORES |
| `test-notepad` | NOTEPAD: Tippen, DEL, der Speicherdialog, OPEN mit nur `.TXT`-Dateien |
| `test-notepad-big` | NOTEPADs Textfenster in einem Z80-Emulator (`tools/notepad_check.py`): eine 12K-Datei mit eigener Datenbank, Bewegen, Tippen und SAVE, Text bis 16K, und 4K auf einem 6128 |
| `test-data` | (576K) NOTEPAD mit 15K Text bis Zeile 33, dazwischen MARKDOWN, dann NOTEPAD zurück zum Anfang: die Zeilen aus seiner eigenen Datenbank |
| `test-paint` | PAINT: Linie, Rechteck, Füllen, die Menüs, dann die Ebenen (eine ausblenden), OTHER DRIVE und EXPORT .SCR nach B: ("SAVED AS .SCR") |
| `test-scr` | PAINTs EXPORT .SCR (sein Code in einem Z80-Emulator, `tools/scr_check.py`) wird in BASIC von Diskette geladen, mit der Palette; jedes Pixel des Bildschirms wird geprüft |
| `test-about` | HILFE > ÜBER V-OS |
| `test-music` | MUSIC von `music.dsk` in B:: ein Song spielt und läuft hinter dem Launcher weiter |

| | |
|---|---|
| `test-markdown` | MARKDOWN von `markdown.dsk` in B:: `README.MD` geöffnet, die Seite mit ^P, einen Bildschirm weiter, zurück zum Text |
| `test-open` | Öffnen nach Typ im Commander: `README.MD` in MARKDOWN von B:, eine `.TXT` in NOTEPAD von A: |
| `test-sysinfo` | SYSINFO aus dem Commander: Version, API, Speicher, freie K auf A: und B:, ^R, ^Q. Mit `SYSINFO_RAM=576` mit einer 512K-Erweiterung |
| `test-clip` | die Zwischenablage: Markieren mit SHIFT, ^C und ^V in NOTEPAD, ^A, ^X, ^V, dann ^V in MARKDOWN |
| `test-webget` | von Hand, im CPCEMU von `make linux` (M4-Karte; braucht einen Bildschirm und das Internet): WEBGET lädt `example.com/index.html`. Schließe CPCEMU mit F12, wenn SAVED erscheint; dann wird `INDEX.HTM` auf der Diskette gesucht |
| `test-reader` | READER von `reader.dsk` in B: öffnet `INDEX.MD`, TAB wählt einen Link, RETURN öffnet `ABOUT.MD`, DEL geht zurück, und ein `https://`-Link gibt eine Meldung |
| `test-db` | die Datenbank-Maschine in einem Z80-Emulator (`tools/db_check.py`): tausende zufällige Aufrufe auf vier Tabellen, gegen ein Modell geprüft, die Datei Seite für Seite (`tools/vdb.py`), bis die Diskette voll ist |
| `test-db-disc` | `DB.SYS` mit dem Dateisystem des Kernels auf einem DSK-Abbild (`tools/db_disc_check.py`) |
| `test-streets` | STREETS von `streets.dsk` in B:: ein neues `STREETS.VDB` mit 24 Straßen, Suchen, eine neue Straße |
| `test-database-z80` | DATABASE in einem Z80-Emulator, die Datenbank in Python und der Bildschirm als Text: zwei Tabellen, 47 Datensätze aus dem Formular, die Reihenfolge ohne Groß-/Kleinschreibung und Akzente, Suchen, eine Änderung, ein Löschen, Blättern |
| `test-database` | DATABASE in Caprice32: eine neue Datenbank auf B:, eine Tabelle, drei Datensätze aus dem Formular, die Liste, Suchen, eine Änderung |
| `test-calc-sys` | die Formel-Engine `CALC.SYS` in einem Z80-Emulator (`tools/calc_check.py`): feste Formeln und 40.000 zufällige, Zahlen und Texte, mit einem Modell in Python verglichen |
| `test-sheet-z80` | SHEET in einem Z80-Emulator mit `CALC.SYS` selbst und dem Bildschirm als Text: Werte, `#ERR`, Zellen, die einander benutzen, Blättern, SAVE und OPEN, eine UTF-8-Datei, `BUDGET.VSH`, ein volles Blatt |
| `test-sheet` | SHEET in Caprice32: Zahlen und eine Formel getippt, `CALC.SYS` von A:, das Blatt auf B: gespeichert |
| `test-kbds` | TASTATUREN: ALLE SECHS in Setup, dann in NOTEPAD die Tasten `:` und Z mit jeder Tastatur: `:z`, `:ζ`, `mw`, `ñz`, `öy`, `òz` |
| `test-charset`, `test-reader-utf8` | UTF-8-Dateien auf Griechisch, Französisch, Spanisch, Deutsch und Italienisch in NOTEPAD und in READER |
| `test-utf8` | der Code von `VX_UTF8` in einem Z80-Emulator, gegen `tools/voscs.py` |
| `test-styles` | STYLES.VBN (`blocks/examples/styles.vbt`) aus dem Commander: normal, fett, kursiv, unterstrichen und alle drei, mit dem STYLE-Block |
| `test-music-block` | mit 576K: TUNE.VBN (`blocks/examples/tune.vbt`) startet MUSIC (von B:) hinter sich mit AGANAMEM.MUS; F6 zeigt „> TUNE“ und MUSIC, und RETURN holt MUSIC nach vorn: A:AGANAMEM, PLAYING. Mit `MBLOCK_RAM=128`: „ERST EIN ANDERES PROGRAMM SCHLIESSEN“ |
| `test-amsdos`, `test-reboot`, `test-system` | V-OS-Dateien in BASIC, Neustart, SYSTEM-Diskette |

**Neues Skript:**

1. Füge ein `IF TEST_SCRIPT == n` am Anfang der Kette in `test_script.asm` hinzu, mit `ELSE` vor dem nächsten und einem weiteren `ENDIF` am Ende.

2. Füge im Makefile ein Ziel hinzu, wie `test-sdk`.

Jede Taste wird als `dw from, to : db row, bit` der Tastatur geschrieben. Die Tastennummer ist Reihe·8 + Bit, z. B. RETURN = 18 = Reihe 2, Bit 2.

**Ohne Emulator:** Der Code eines Programms läuft auch auf einem Z80 in Python (das Paket `z80`). Du lädst die `.BIN` bei `#4000` und lässt die API-Aufrufe sofort zurückkehren (oder tun, was nötig ist, z. B. Dateien in einem Dictionary). Dann rufst du init und event auf und liest den Speicher. Das ist bei Fehlern in Zahlen, Dateien und Sprüngen viel schneller als Screenshots. Die Adressen der Labels kommen von `rasm … -s -sl` (eine `.sym`-Datei).

## Bei jedem Meilenstein

1. Code, und ein Test in Caprice32: neu oder vorhanden, mit Screenshots.
2. Die Tests, die die Änderung berührt.
3. Ein Abschnitt in `MILESTONES.md` und `MILESTONES.el.md`, und ein Abschnitt „Υλοποίηση M…“ (Umsetzung) in `docs/REQUIREMENTS.md`.
4. **Das Handbuch in allen sechs Sprachen aktualisieren** (`docs/manual/` auf Griechisch, `docs/manual/<code>/` für en, fr, es, de, it, und `sdk/README.md`, `sdk/README.<code>.md`, wenn sich die API ändert), dann `make docs` für alle PDFs.
5. Commit und Push.